PERFORMANCE-MESSPLAN

Messungen für Netlify, Railway und PostgreSQL

Konkreter Ablauf zum Vergleich des aktuellen Stands mit den Performance-Änderungen aus PR #112.

Betriebsannahmen bestätigt: Netlify und Railway liegen in derselben Region. Die App verwendet den installierten pgBouncer-Pooler. Diese Einstellungen bleiben während aller Messungen unverändert.

Ziel und Vergleich

Der Plan soll zeigen, welche Änderung die wahrgenommene Ladezeit, die Serverlaufzeit, die Datenbanklast und den Bildtransfer tatsaechlich verbessert. Ein Ergebnis gilt erst dann als belastbar, wenn Ausgangsstand und PR-Stand unter gleichen Bedingungen gemessen wurden.

Bereich	Änderung im PR	Was die Messung zeigen soll
Dashboard	Rückfall-Poll startet bei 30 Sekunden, steigt bis 120 Sekunden und pausiert im Hintergrund.	Weniger Refresh-Aufrufe und DB-Arbeit bei ausgefallenem Realtime-Kanal.
Datenzugriff	Viewer und Chronologie sowie Archiv-Verweise laden unabhaengig parallel.	Niedrigere Serverzeit bei unveränderter Query-Anzahl.
Chronologie	Erste 50 Karten im DOM; weitere per Knopf. Volltexte werden erst beim Oeffnen oder in Scrollnaehe geladen.	Schnellerer erster Aufbau, weniger initiales HTML und weniger Markdown-Arbeit.
Client-Bundle	Ably wird bei aktivierter Realtime-Funktion als eigener Chunk nach der Hydrierung geladen.	Kleineres initiales JavaScript und weniger Arbeit vor der ersten Interaktion.
Bilder	Berechtigungspruefung und privater Browser-Cache bleiben erhalten.	Bytegröße und Cache-Treffer messen; keine ungeschuetzte gemeinsame CDN-Auslieferung.

Wichtige Abgrenzung

Die Chronologie rendert anfangs weniger Karten, laedt für Filter aber weiterhin die Event-Metadaten. Das ist progressive Darstellung und noch keine Datenbank-Paginierung. Der Messplan prueft deshalb sowohl DOM und HTML/RSC-Transfer als auch die Serverzeit. Bleibt der Event-Payload der Engpass, ist echte serverseitige Cursor-Paginierung ein eigener Folgeschritt.

1. Testbedingungen festhalten

Vor dem ersten Lauf eine Messkarte anlegen. Ausgangs- und PR-Lauf muessen dieselbe Netlify-Region, dieselbe Railway-Postgres-Instanz, dieselbe Datenbankgröße und denselben Pooler verwenden. Auch Node-/Next-Version, Umgebungsvariablen, Limits, Authentifizierungsmodus und Testdaten bleiben gleich.

Feld	Einzutragen
Zeitfenster und Zeitzone	Datum, Start/Ende, Europe/Berlin oder UTC
Netlify	Site, Deploy-ID, Branch/Commit, Region, Observability-Ansicht oder Function Metrics
Railway	Projekt/Umgebung, App-Service, Postgres-Service, Deploy-IDs
Datenbank	Snapshot/Stand, Zeilenanzahl der Haupttabellen, Pooler-Endpunkt bestätigt
Browser/Gerät	Chrome-Version, Desktop oder Mobile-Profil, Netzwerkprofil
Cachezustand	Kaltstart, warmer CDN-/Browsercache, neuer oder bestehender Login
Code	Baseline-Commit und PR-Commit; pro Lauf nur eine Codebasis

2. Repraesentative Seiten und Anfragen

Szenario	Vorgehen	Aufzeichnen
/	Anonym und angemeldet, jeweils direkter Aufruf und Navigation.	TTFB, LCP, HTML/RSC-Bytes, Function-Dauer.
/chronologie?scope=all	Einmal frisch laden, danach Filter, Sortierung und Weitere-Ereignisse-Knopf.	DOM-Knoten, Karten im initialen DOM, Transferbytes, Zeit bis nutzbare Filter.
Chronologie-Sessions	Eine Session im ersten Viewport oeffnen; weitere durch Scrollen nachladen.	Anzahl Detail-Requests, Antwortbytes und Ladezeit je Detail.
/characters/{slug}	Eine reprasentative Charakterseite mit Portrait auf Desktop und Mobile.	JS-Bytes, Portraitbytes, LCP, Bildrequests.
/archive/{slug}	Ein Eintrag mit vielen Links und Backlinks.	Serverzeit, DB-Query-Zeit und Gesamtzeit.
/api/content-images/{id}	Ein freigegebenes Bild zweimal laden; danach Berechtigungsfall für Entwurf prüfen.	Status, Bytes, Browser-Cache-Treffer, Funktionsaufrufe.

GET-Messungen bevorzugen. Für Tests mit Formularen und Schreibaktionen ausschliesslich separate Testkonten und Testdaten verwenden.

3. Kennzahlen nach Ebene

Pro Szenario mindestens p50 und p95 erfassen; p99 bei ausreichender Stichprobe. Werte immer gemeinsam mit Zeitraum, Cachezustand und Request-Anzahl dokumentieren.

Ebene	Kennzahlen	Wo messen
Browser und UX	LCP, INP, CLS; Lighthouse-TBT als Labornaeherung für INP; Zeit bis erste Karte und bis bedienbare Filter.	Chrome DevTools Performance / Lighthouse; reale Nutzerdaten nur falls bereits vorhanden.
HTTP und Netlify	TTFB, Gesamtdauer, p50/p95/p99, 4xx/5xx, Anzahl Function-Aufrufe, Function-Dauer und Cache-Status.	Netlify Observability oder Function Metrics, Deploy-Logs, Browser Performance.
Railway App-Service	CPU, RAM, Netzwerk, Restart-/Crash-Signale während exakt desselben Zeitfensters.	Railway Service Metrics.
PostgreSQL	Query-Aufrufzahl und Laufzeit, aktive/idle Verbindungen, Wartezeit, Timeouts, CPU/RAM und Buffer-Cache-Treffer.	Railway DB Query Statistics / Metrics, Pooler-Metriken und Query-Logs falls vorhanden.
Timeline	Initiale Event-Transferbytes, Karten/DOM-Knoten, Renderdauer, Detailrequests pro Scrollstrecke.	Chrome DevTools Network und Performance; DOM-Inspektion.
Bilder und Client JS	Bildbytes und Wiederholungsrequests; initiale JS-kB (transferiert und gzip/brotli), Ably-Chunk und Long Tasks.	Chrome DevTools Network, Coverage, Performance; Response-Header.

Vorlaeufige Feldziele

- Core Web Vitals am 75. Perzentil: LCP höchstens 2,5 Sekunden, INP höchstens 200 ms und CLS höchstens 0,1.
- Lighthouse kann INP nicht direkt messen; TBT dient im Labor nur als Proxy und wird nicht als gleichwertiger INP-Wert ausgegeben.
- Für Netlify und DB gibt es vorab keinen belastbaren absoluten Grenzwert. Zuerst Baseline erfassen, danach Prozentveränderung und Fehlerquote vergleichen.
- Ein schnellerer Mittelwert reicht nicht, wenn p95/p99 oder die Fehlerrate deutlich schlechter werden.

Netlify Function Metrics kann je nach Tarif durch Observability ersetzt sein. Railway erfasst Infrastrukturwerte; Request-Latenz und anwendungsspezifische Kennzahlen muessen separat aus Browser, Netlify oder Telemetrie kommen.

4. Messablauf

Schritt	Durchfuehrung
A. Baseline	Baseline-Deploy auswählen. Pro Route 5 Browser-Läufe mit gleichem Viewport und Cachezustand. Ersten Lauf separat als kalt markieren, die folgenden als warm.
B. PR-Stand	Nach erfolgreichem Preview-Deploy dieselben Routen, Konten und Daten mit derselben Browser-/Netzwerk-Konfiguration wiederholen.
C. Laststufen	Mit k6 oder einem vergleichbaren Werkzeug 1, 5 und 10 parallele Leser schrittweise prüfen. Pro Stufe 5 Minuten nach einer kurzen Aufwärmphase. 25 Leser erst nach stabilen niedrigeren Stufen.
D. Authentifizierte Lesewege	Eine Session mit Testkonto anmelden und private Seite/Bild-Route nur für dieses Konto messen. Session-Cookie als Secret behandeln; nie in Konsolen- oder Ergebnisdateien schreiben.
E. Details und Scroll	Chronologie laden, initiale Karten zaehlen, Scrollen bis zum nächsten 50er-Block, Session-Panel oeffnen und manuelles Ereignis aufrufen. Network-Aufzeichnung exportieren.
F. Wiederholung	Jeden Lastlauf 3-mal wiederholen. Dazwischen mindestens 2 Minuten warten. Deploy-Marker und Zeitfenster aus Netlify/Railway für jeden Lauf festhalten.
G. Auswertung	Baseline und PR nach identischem Lastprofil nebeneinanderstellen. p95, Fehlerrate und Ressourcenverlauf zuerst vergleichen; danach Ursache einer Verbesserung bestimmen.

Cache kalt und warm trennen

- Kalt: neues privates Browserfenster, kein vorheriger Seiten- oder Bildrequest. Kaltwerte nicht mit wiederholten Besuchern vermischen.
- Warm: exakt denselben GET erneut ausführen und Browser-Cache sowie Status 304/Cache-Treffer dokumentieren.
- Beim Bildproxy ist der Cache privat und auf eine Stunde begrenzt. Teste Wiederverwendung nur im selben Browserprofil und nicht mit einem anderen Konto.
- HTML-/RSC-Auslieferung separat messen: ein schneller warmer Browsercache beweist keine kürzere Serverfunktion.

Lastprofil und Abbruchkriterium

Der Lastgenerator erzeugt vorwiegend lesende Anfragen: 70 Prozent Seitenaufrufe, 20 Prozent Detail-GETs, 10 Prozent Bilder. Sofort stoppen, wenn 5xx, DB-Pool-Timeouts oder Railway-Ressourcen in einen anhaltenden Grenzbereich laufen. Kein Schreibtest gegen produktive Inhalte.

5. Hypothesen für diesen PR

Änderung	Vergleich	Erfolgssignal	Möglicher Gegeneffekt
Fallback-Poll	Ably getrennt deaktivieren/unerreichbar simulieren; 30-120-Sekunden-Verhalten beobachten.	Weniger Refreshes und SQL-Aufrufe je Stunde; Rückkehr zum Tab aktualisiert sofort.	Dashboard-Inhalte koennen ohne Realtime bis zu 120 Sekunden alt sein.
Parallele DB-Arbeit	Chronologie und Archivdetail mit derselben DB; serverseitige Gesamtzeit sowie DB-Einzelquery-Zeit.	Kürzere Requestzeit, Querydauer selbst bleibt gleich.	Mehr gleichzeitige Client-Verbindungen; Pooler-Auslastung mitbeobachten.
Timeline-Rendering	Initiale Karten und DOM-Knoten vor/nach; HTML/RSC-Transfer getrennt.	Weniger initiale Karten, schnellerer DOM-Aufbau und kürzere Main-Thread-Arbeit.	Event-Metadaten für Browserfilter werden weiterhin voll geladen.
Lazy Detailtexte	Anzahl und Bytes der Detailantworten beim ersten Laden, Scrollen und Oeffnen.	Kein Volltext-HTML im ersten Payload; nur sichtbare/geöffnete Detailtexte werden abgerufen.	Zusaetzlicher Roundtrip beim Oeffnen; Fehlerzustand und Wiederholen mitprüfen.
Ably-Chunk	Initiales JS bei aktivierter und deaktivierter Realtime-Funktion vergleichen.	Initialer Chunk kleiner; Ably nicht im kritischen Hydrierungspfad.	Ably wird nach Hydrierung dennoch nachgeladen, wenn aktiviert.
Content-Bilder	Erster und wiederholter Request; Antwortgröße, Cache-Control, CDN- und Function-Aufrufe.	Wiederholter Browserrequest spart Bytes und R2-Fetch.	Aktueller privater Endpoint kann nicht bedenkenlos public am CDN gecacht werden.

Bild-CDN: sichere nächste Stufe

Netlify Image CDN optimiert Formate und cached identische Transformationen an der Edge. Die aktuelle Content-Bildroute prueft aber bei jedem Zugriff Rechte und liefert Entwurfsbilder über denselben ID-Pfad aus. Public Edge-Caching dieser URL koennte ein Bild nach einem Wechsel von veröffentlicht zu Entwurf weiter ausliefern.

- Im Messlauf deshalb keinen Public-Cache-Header auf /api/content-images setzen und keine Authentifizierung entfernen.
- Wenn Bildbytes ein Engpass sind: separate, nur für veröffentlichte Inhalte nutzbare Derivate mit eigener URL und widerrufbarer Versionskennung prüfen.
- Vor Aktivierung prüfen, dass Sichtbarkeitswechsel den Cache-Key wechseln oder eine CDN-Purge ausloesen.
- Netlify erlaubt Remote-Transformationen erst nach Domain-Freigabe. Beliebige Portrait-Hosts nicht pauschal freigeben.

6. Entscheidung nach dem Vergleich

- Verbesserung bestätigt: p95 der betroffenen Route sinkt in mindestens 2 von 3 Läufen; Fehlerrate und p99 verschlechtern sich nicht deutlich.
- Client-Fix bestätigt: initiale JS-Bytes/Long Tasks sinken oder späterer Ladevorgang ist aus der Hydrierungszeit herausgenommen; gemessene LCP/Interaktion darf nicht schlechter werden.
- Timeline-Fix bestätigt: erste 50 Karten erscheinen frueher und der initiale HTML/RSC-Payload enthaelt keine gerenderten Session-/Freitext-Volltexte.
- Wirkung nicht belegt: Ergebnis liegt innerhalb der Streuung. Nicht als Performancegewinn melden; Testdaten, Warmup und Stichprobe verbessern.
- Regression: eine Rücknahme oder gezielte Nachbesserung an der verursachenden Stelle vornehmen und dieselben Schritte wiederholen.

Messprotokoll zum Ausfuellen

Lauf	Baseline	PR #112
Commit / Deploy-ID		
Szenario, Cache, Gerät		
Requests / VUs		
p50 / p95 / p99		
5xx / Timeout-Rate		
Netlify Function p95		
DB Query p95 / Pool-Wait		
Railway CPU / RAM		
Initial JS / HTML-RSC kB		
LCP / INP / CLS oder TBT		
Bildbytes / Cache-Treffer		
Beobachtung / Entscheidung		

Die Regions- und Pooler-Fragen gelten als beantwortet und werden nicht erneut als Fehlerursache getestet. Sie sind konstante Randbedingungen für den A/B-Vergleich.

7. Quellen und Hinweise

Die folgenden offiziellen Seiten erklaeren, welche Messwerte in den jeweiligen Oberflaechen verfügbar sind. Der Zugriff auf Funktionen und Aufbewahrungszeit kann vom Netlify-Plan abhaengen.

Netlify Function Metrics - Aufrufe, Fehler, Dauer sowie p50, p95 und p99; mit Tarifhinweis und Weiterleitung zu Observability.

Netlify Observability - Produktionssicht auf Requests, Cache-Status, regionale Latenz und Function-Ausführung.

Railway Metrics - CPU, Speicher, Disk und Netzwerk pro Service; Railway weist darauf hin, dass App-Latenz separat instrumentiert werden muss.

Railway PostgreSQL - PostgreSQL-Service, Backups und Beobachtbarkeit.

Railway: Postgres-Metriken - Datenbank-Verbindungen, Cache-Treffer, Query-Statistiken und Tabellenmetriken im Dashboard.

web.dev: Web Vitals - Core Web Vitals und Labor-/Felddaten; TBT als Labornaeherung für INP.

Grafana k6: Thresholds - Prozentile und explizite Kriterien für HTTP-Dauer und Fehlerquote.

Interpretation

Dieser Plan definiert Messungen und Vergleichskriterien, aber behauptet keinen bereits erreichten Zahlengewinn. Den Performancegewinn erst nach Ausführung mit denselben Deploy-, Datenbank- und Cachebedingungen in PR #112 dokumentieren.

Code-Stand: Version 2.2.1. Die CI bleibt gemäß Projektvorgabe manuell und wird von diesem Messplan nicht gestartet.